DISCRETE MATHEMATICS

Project

Universidad Javeriana, Cali
Semester 2, 2026

1. Context

On the first term we have studied propositional logic. We saw operators as $\wedge$, $\vee$, $\implies$, $\iff$ and $\neg$. A formula in propositional logic is either a variable (p,q,r,etc), $\neg(\phi)$ where ϕ is a formula or $(\phi)\square(\beta)$ where ϕ, β are formulas and $\square \in \{\wedge, \vee, \implies, \iff\}$. Thus, we get formulas like

$$\neg((p) \vee (q))$$

$$(\neg(p)) \wedge (\neg(q))$$

$$((p) \iff (q)) \implies (r)$$

$$(p) \vee (\neg(p))$$

among others.

In the second term we have studied some set theory. We saw operators like $\cap$, $\cup$ and $\overline{X}$. A set formula is either some set (X, Y, etc), $\overline{(\phi)}$ where ϕ is some formula, or $(\phi)\square(\beta)$ for ϕ, β formulas and $\square \in \{\cap, \cup\}$. Thus, we get formulas like

$$(X) \cap (\overline{(Y)})$$

$$(X) \cup (Y)$$

among others.

In basic arithmetic, we know of basic operators like $-$, $+$ and $*$. An arithmetic formula is either a variable (x, y, z, etc), $-(\phi)$ where ϕ is some formula, or $(\phi)\square(\beta)$ where ϕ, β are formulas and $\square \in \{-, +, *\}$. Thus, we get formulas like

$$(x) + ((z) * (-(y)))$$

$$(x) - (z)$$

among others.

What is the common thread among those 3 types of formulas?

2. Statement

Let $\mathfrak{F}$ be the set of all formulas. The set $\mathfrak{F}$ is built according to the following rules:

1. If p is a *variable*, then $p \in \mathfrak{F}$
2. If $\phi \in \mathfrak{F}$ and $\circ$ is a *unary operator*, then $\circ(\phi) \in \mathfrak{F}$
3. If $\phi, \beta \in \mathfrak{F}$ and $\square$ is a *binary operator*, then $(\phi)\square(\beta) \in \mathfrak{F}$

We shall assume that we are working on some domain U, in which binary operators are functions $U \times U \to U$, and unary operators are functions $U \to U$.

You shall implement a function that, given a string representing a formula, creates an expression tree.

`https://en.wikipedia.org/wiki/Binary_expression_tree`

Some other function that, given an expression tree and a valuation, evaluates the formula and outputs the result. In this project, valuations are maps `str` $\to (U \to U)$ for unary operators, `str` $\to (U \times U \to U)$ for binary operators, and `str` $\to U$ for variables (notice this is not necessarily the general use of the concept).

Notice there are some 'blank spaces' (space and tab) that should be ignored while parsing. You may assume that all formulas are well formed as described above. You may also assume that names of variables or operators do not contain blank spaces or parenthesis ())()(())).

3. Technical Deliverables

Notice that the channels to be used to deliver the project are to be explained by each teacher of the practical sessions of the course.

3.1 Code 45%

1. The implementation of a function `expression_tree` : `str` $\rightarrow$ `Formula` that receives a formula as a string and generates an expression tree.

2. The implementation of a function `__str__` : `Formula` $\rightarrow$ `str` that receives an expression tree and generates its corresponding string representation.

3. The implementation of a function `eval` : (`Formula` $\times$ `Valuation`) $\rightarrow U$ that receives an expression tree and a valuation and evaluates the formula represented. The string should basically be the input string with no blank spaces.

4. A function `count` : `Formula` $\rightarrow$ (`str` $\rightarrow \mathbb{N}$) that receives an expression tree and counts the appearances of variables, unary operators and binary operators in the formula represented.

All this are to be implemented to follow the interface shown in
`https://github.com/osvarp/Discrete_Math_Project20262/`
More specifically, implement the abstract class `Formula` and the function `expression_tree` in `solution.py`. Notice the interfaces are written in `interface_function.py`.
'Blank_Spaces' is defined in `helpers.py`.

3.2 Report 20%

A document must be submitted that briefly explains the ideas that led to the implementation of the solution. It will be evaluated based on its coherence with the solution presented and how well it explains the ideas used.

The main idea is to be able to completely understand the codes with this report. However, the right is reserved for calling students to explain their solutions.

3.3 Questionnaire 35%

A Questionnaire will be held. It is still in discussion the modality and date where it will take place. This will be elaborated further in another moment.

4. Implementation Constraints

- **Use of artificial intelligence is forbidden:** Any use of AI in the completion of this project will result in its cancellation and possible disciplinary action.
- **Good programming practices:** The use of `continue`, `break`, or `pass` is not permitted. If a function returns a value, there should only be one return, which must be at the end of the definition. The exception is `pass` for defining abstract methods and functions.
- **The use of native functions such as `eval` or external libraries is forbidden**: The implementation must be carried out using only native Python functionalities. The objective is for students to understand all the functions and definitions involved. Do not confuse the native 'eval' to the 'eval' to be defined for Formulas.